CS 4/5782 Final Report

Jason Dong, Sennet Senadheera, and Dalton Luce*
Cornell University
Ithaca, NY, USA
{jd876,sas639,dcl252}@cornell.edu

1 Introduction

Fine-tuning is a common task used to adapt large pre-trained models to specific downstream applications. By updating the model's weights on a smaller dataset, this process preserves general linguistic knowledge while specializing the model for a new domain. However, the naive approach of adjusting all model weights induces prohibitively high computational and memory costs, often requiring specialized hardware for even modest tasks. Alternative methods such as prefix tuning address this by training fewer parameters but ultimately take away from the input sequence length, while adapter tuning introduces bottleneck layers that add latency during inference time.

To address these limitations, Hu et al. propose "LoRA: Low-Rank Adaptation of Large Language Models," a method that freezes the pre-trained weights (W_0) and injects trainable low-rank decomposition matrices into Transformer layers [1]. Specifically, LoRA represents the weight update as $W = BA$, where the rank r is much smaller than the hidden dimension. When applied to GPT-3 175B, this approach dramatically reduces the number of trainable parameters by up to 10,000 times and GPU memory requirements by up to 3 times, while preserving the model's original context window and inference speed.

2 Chosen Result

For our re-implementation, we target the RoBERTa-base performance metrics documented in Table 2 of the original paper (Hu et al., 2021) [1]. Specifically, we reproduce the comparative analysis between Full Fine-Tuning and LoRA (r=8) on two key benchmarks from the General Language Understanding Evaluation (GLUE) suite: SST-2 (Sentiment Analysis) and MNLI (Natural Language Inference) [2].

Model & Method	# Trainable Parameters	MNLI	SST-2
RoB_{base} (FT)*	125.0M	**87.6**	94.8
RoB_{base} (BitFit)*	0.1M	84.7	93.7
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$

Figure 1: Chosen results from Table 2 of [1].

This result was chosen as it validates the core LoRA hypothesis: that adapting a model via low-rank decomposition can match or exceed the performance of full-parameter fine-tuning. In the paper, LoRA achieves an accuracy of 94.9% on SST-2 and 87.5% on MNLI, performing on par with full fine-tuning while training only 0.3M parameters (approximately 0.24% of the 125M total parameters in RoBERTa-base). Successfully reproducing these figures shows that our implementation works correctly and that substantial compute and memory savings can be achieved without significantly reducing model accuracy.

3 Methodology

We implement LoRA from scratch, applying it to RoBERTa-base (125M parameters, 12 layer transformer). The core module, `LoRALinear`, augments selected `nn.Linear` layers with two additional trainable matrices: $A \in \mathbb{R}^{r \times k}$ (initialized with Kaiming uniform initialization, following the paper's setup) and $B \in \mathbb{R}^{d \times r}$ (initialized to zero, ensuring $\Delta W = BA = 0$ at the start of training). The forward pass computes:

*https://github.com/da-luce/cs5782_final_project/

$$h = W_0 x + \frac{\alpha}{r}(BAx)$$

where α is a fixed scaling constant (set to 16 as per the paper).

LoRA is injected into the query and value projection matrices across all 12 transformer attention blocks, matching the configuration in Table 2 of the original paper. All base model weights are frozen; only the LoRA matrices and the randomly-initialized task-specific classification head are trained.

We evaluate on two GLUE benchmark tasks from the paper: SST-2 (binary sentiment classification) and MNLI (3-class natural language inference), loaded via HuggingFace Datasets [3]. Due to limited compute resources (a single NVIDIA H100 GPU on Google Colab), we train on random subsets of 50,000 training examples for each task over 3 epochs (v.s. the full 67K/393K used in the paper). For evaluation, we use 872 validation examples from SST-2 and 5,000 validation examples from MNLI, using the official `validation` and `validation_matched` splits respectively.

The primary metric that we evaluate is the classification accuracy in the validation set, which matches the evaluation protocol in the paper. Additionally, we report the count and percentage of total parameters, training time, peak GPU memory usage, and checkpoint size on disk to characterize the efficiency benefits of LoRA.

We use rank $r = 8$, $\alpha = 16$, and dropout 0.1, exactly matching Table 2 of the paper. Learning rates are 2×10^{-4} for LoRA and 2×10^{-5} for full fine-tuning, with batch size 8. To make experiments feasible on a single GPU, we make two adjustments relative to the paper:

(1) We use the training subsets described above
(2) We truncate sequences to 128 tokens rather than RoBERTa's default of 512, since the vast majority of SST-2 sentences and MNLI pairs fit within this limit (See Appendix B).

We also conduct a rank ablation study over $r \in \{2, 4, 8, 16\}$ to characterize the accuracy–efficiency tradeoff.

4 Results & Analysis

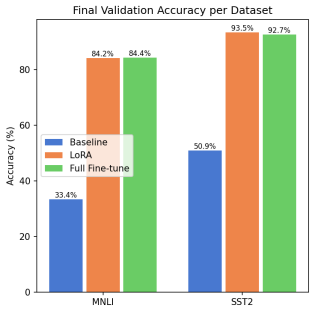

Figure 2: Bar chart summarizing our validation accuracy. We include the baseline (no fine-tuning) to show that without any fine tuning, the model is effectively guessing at random.

Our results are summarized in Figure 2. A detailed table of results can be viewed in Table 1, Appendix A. We do not match the absolute accuracy of the paper, likely due to training on only a limited subset of the data. However, we do achieve comparable relative performance trends between the baseline and LoRA. Specifically, our results demonstrate that the LoRA-adapted model consistently tracks within a narrow margin of the full fine-tuned baseline for the MNLI task, and even narrowly exceeds the full fine-tune accuracy on the SST-2 sentiment analysis task. Additionally, the peak VRAM and training time were significantly reduced, with LoRA requiring only a fraction of the memory overhead of full fine-tuning.

Sweeping $r \in 2, 4, 8, 16$, validation accuracy remains essentially flat (approximately 78–80% on MNLI and 92–93% on SST-2) with no consistent improvement as rank increases. This stability suggests that SST-2 and MNLI are sufficiently simple classification tasks that even a rank-2 update captures the full adaptation signal needed by RoBERTa-base, and that additional rank capacity goes unused (Figure 3).

5 Reflections

Our results closely corroborate the paper's central claim: LoRA matches full fine-tuning with under 1% of trainable parameters. Our rank ablation extends this finding further, showing that even $r = 2$ achieves competitive accuracy on both tasks, suggesting the adaptation signal for these GLUE benchmarks is intrinsically low-dimensional. An unexpected finding was that LoRA acted as an implicit regularizer on MNLI: full fine-tuning showed validation loss divergence by epoch 2, while LoRA's constrained update space kept validation loss stable throughout training. Together, these results suggest that LoRA's low-rank bottleneck is not merely a parameter reduction trick, but a structural prior that shapes how the model adapts, i.e. concentrating updates in the directions that matter and preventing overfitting in the directions that do not (Figure 4).

References

[1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *CoRR* abs/2106.09685 (2021). arXiv:2106.09685 https://arxiv.org/abs/2106.09685

[2] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. *CoRR* abs/1804.07461 (2018). arXiv:1804.07461 http://arxiv.org/abs/1804.07461

[3] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2019. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *CoRR* abs/1910.03771 (2019). arXiv:1910.03771 http://arxiv.org/abs/1910.03771

A Appendix: Additional Results

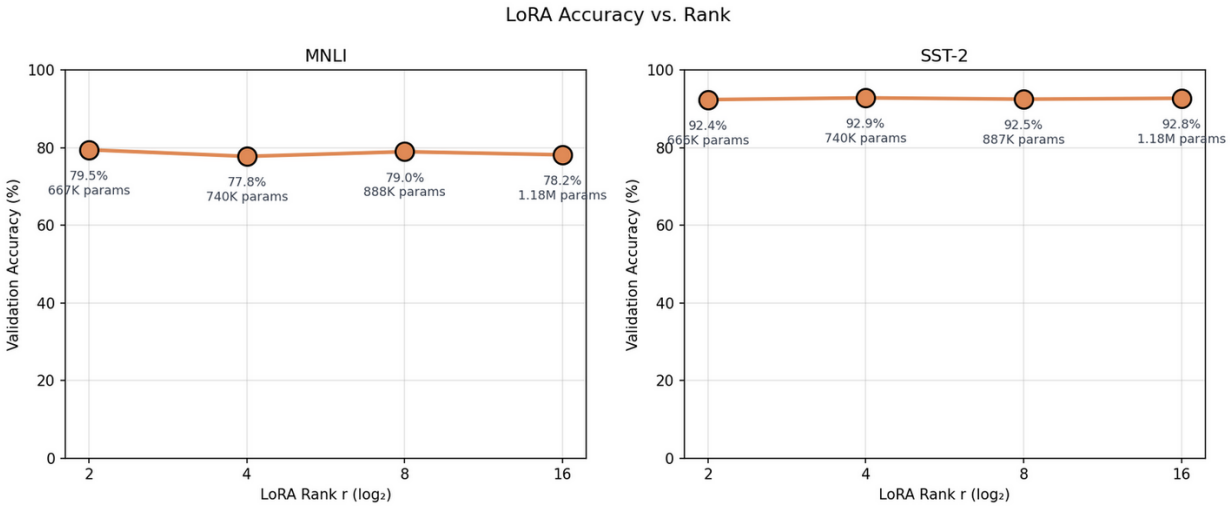

Figure 3: Validation accuracy vs. LoRA rank $r \in \{2, 4, 8, 16\}$ for MNLI (left) and SST-2 (right), with parameter counts shown per point. Accuracy remains stable across all ranks for both tasks (~ 78–80% on MNLI, ~ 92–93% on SST-2), supporting the paper's claim that low-rank updates are sufficient for task adaptation.

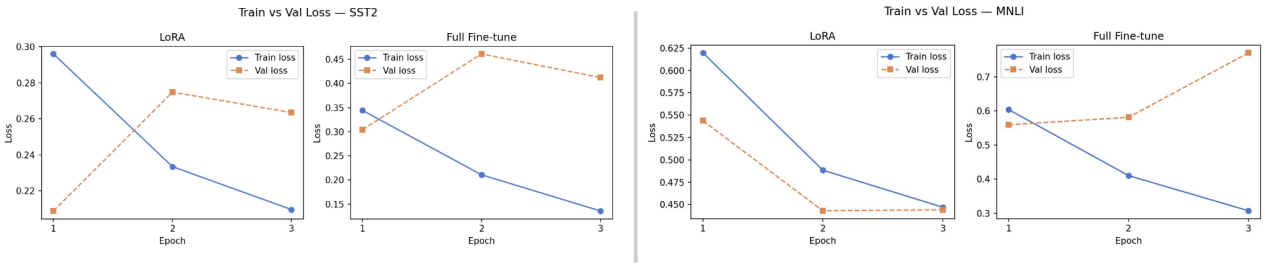

Figure 4: Training and validation loss across 3 epochs for SST-2 (left) and MNLI (right) $N = 50{,}000$. On MNLI, LoRA ($r = 8$) effectively mitigates overfitting compared to full fine-tuning, which shows significant validation loss divergence by epoch 2.

Method	Dataset	Accuracy	Params	%	Time	Ckpt	Peak VRAM
Baseline (untrained)	MNLI	33.4%	124.6M	100.00%	—	0.0M	543.4M
Full fine-tuning (paper)	MNLI	87.6%	—	—	—	—	—
Full fine-tuning (ours)	MNLI	84.4%	124.6M	100.00%	9.2m	475.5M	2135.4M
LoRA $r = 8$ (paper)	MNLI	87.5%	—	—	—	—	—
LoRA $r = 8$ (ours)	MNLI	84.2%	0.9M	0.71%	7.0m	476.7M	1045.2M
Baseline (untrained)	SST-2	50.9%	124.6M	100.00%	—	0.0M	543.3M
Full fine-tuning (paper)	SST-2	94.8%	—	—	—	—	—
Full fine-tuning (ours)	SST-2	92.7%	124.6M	100.00%	8.9m	475.5M	2135.3M
LoRA $r = 8$ (paper)	SST-2	94.9%	—	—	—	—	—
LoRA $r = 8$ (ours)	SST-2	93.5%	0.9M	0.71%	6.6m	476.7M	1045.2M

Table 1: RoBERTa-base results on GLUE tasks. Paper results from Table 2 of Hu et al. [1]. The original paper reports 0.3M trainable parameters by counting only the LoRA adapter matrices added to the query and value projections, whereas our 0.9M total includes the task-specific classification head necessary for training.

B Appendix: Sequence Length Statistics

Dataset	Median	p_{95}	≤128 tokens
SST-2	11	34	100.0%
MNLI	38	73	99.7%

Table 2: Tokenized sequence length statistics on $N = 50{,}000$ samples using RoBERTa tokenizer. Thus, truncating at 128 tokens preserves nearly all contextual information while optimizing computational efficiency. Created by running `code/misc/analyze_truncation.py`.